# Page 1 | OpenStax Python Programming: Chapter Statements - Background

*Module ID: m00002 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,734*

## Learning Objectives

By the end of this section you should be able to

Name two examples of computer programs in everyday life. Explain why Python is a good programming language to learn.

## Computer programs

A computer is an electronic device that stores and processes information. Examples of computers include smartphones, tablets, laptops, desktops, and servers. Technically, a program is a sequence of instructions that a computer can run. Programs help people accomplish everyday tasks, create new technology, and have fun.

The goal of this book is to teach introductory programming and problem solving. Writing programs is a creative activity, inherently useful, and rewarding! No prior background in computer science is necessary to read this book. Many different types of programs exist, as shown in the illustration below. This book will focus on general purpose programs that typically run "behind the scenes."

Online music streaming Online music streaming; ch 1, video 1

Computers and programs

Exploring further Later chapters of this book show how to write analysis programs using real data. Example libraries that provide access to online streaming services include Spotipy, Pytube, and Pydora. Python-related tools often have the letters "py" in their name.

## The Python language

This book introduces Python, one of the top programming languages today. Leading tech giants like Google, Apple, NASA, Instagram, Pixar, and others use Python extensively.

One reason why Python is popular is because many libraries exist for doing real work. A library is a collection of code that can be used in other programs. Python comes with an extensive Standard Library for solving everyday computing problems like extracting data from files and creating summary reports. In addition, the community develops many other libraries for Python. Ex: Pandas is a widely used library for data analysis.

Another reason why Python is popular is because the syntax is concise and straightforward. The syntax of a language defines how code must be structured. Syntax rules define the keywords, symbols, and formatting used in programs. Compared to other programming languages, Python is more concise and straightforward.

Hello world in Python and Java By tradition, Hello World is the first program to write when learning a new language. This program simply displays the message "Hello, World!" to the user. The hello world program is only one line in Python: print("Hello, World!") In contrast, the hello world program is five lines in Java (a different language). public class Hello { public static void main(String[] args) { System.out.println("Hello, World!"); } } However, conciseness is not the only consideration for which language is used. In different situations different languages may be more appropriate. Ex: Java is often used in Android development.

Counting lines in a file Counting lines in a file; ch 1, video 2

Python vs Java syntax

Favorite song The program below asks for your name and displays a friendly greeting. Run the program and see what happens. In the error message, EOF stands for End of File. Many of the programs in this chapter expect input from the user. Enter your name in the Input box below the code. Run the program again, and see what changes. Copy the following lines to the end of the program: print("What is your favorite song?") song = input() print("Cool! I like", song, "too.") The modified program reads two lines of input: name and song. Add your favorite song to the Input box below your name, and run the program again. The next section of the book will explain how print() and input() work. Feel free to experiment with this code until you are ready to move on.

# Page 2 | OpenStax Python Programming: Chapter Statements - Input/output

*Module ID: m00003 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,340*

## Learning objectives

By the end of this section you should be able to

Display output using the print() function. Obtain user input using the input() function.

## Basic output

The print() function displays output to the user. Output is the information or result produced by a program. The sep and end options can be used to customize the output. shows examples of sep and end.

Multiple values, separated by commas, can be printed in the same statement. By default, each value is separated by a space character in the output. The sep option can be used to change this behavior.

By default, the print() function adds a newline character at the end of the output. A newline character tells the display to move to the next line. The end option can be used to continue printing on the same line.

Example uses of print(). Code Output print("Today is Monday.") print("I like string beans.") Today is Monday. I like string beans. print("Today", "is", "Monday") print("Today", "is", "Monday", sep="...") Today is Monday Today...is...Monday print("Today is Monday, ", end="") print("I like string beans.") Today is Monday, I like string beans. print("Today", "is", "Monday", sep="? ", end="!!") print("I like string beans.") Today? is? Monday!!I like string beans.

Displaying output to the user 1 3 1 Displaying output to the user

The print() function

Do spaces really matter? Spaces and newline characters are not inherently important. However, learning to be precise is an essential skill for programming. Noticing little details, like how words are separated and how lines end, helps new programmers become better.

## Basic input

Computer programs often receive input from the user. Input is what a user enters into a program. An input statement, variable = input("prompt"), has three parts:

A variable refers to a value stored in memory. In the statement above, variable can be replaced with any name the programmer chooses. The input() function reads one line of input from the user. A function is a named, reusable block of code that performs a task when called. The input is stored in the computer's memory and can be accessed later using the variable. A prompt is a short message that indicates the program is waiting for input. In the statement above, "prompt" can be omitted or replaced with any message.

Obtaining input from the user Displaying output to the user; ch 1, video 3

The input() function

Frost poem Write a program that uses multiple print() statements to output the following poem by Robert Frost. Each print() statement should correspond to one line of output. Tip: You don't need to write the entire program all at once. Try writing the first print() statement, and then click the Run button. Then write the next print() statement, and click the Run button again. Continue writing and testing the code incrementally until you finish the program. I shall be telling this with a sigh Somewhere ages and ages hence: Two roads diverged in a wood, and I-- I took the one less traveled by, And that has made all the difference.

Name and likes Write a program that asks the following two questions (example input in bold): What is your name? Shakira What do you like? singing Output a blank line after reading the input. Then output the following message based on the input: Shakira likes singing

# Page 3 | OpenStax Python Programming: Chapter Statements - Variables

*Module ID: m00004 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,143*

## Learning objectives

By the end of this section you should be able to

Assign variables and print variables. Explain rules for naming variables.

## Assignment statement

Variables allow programs to refer to values using names rather than memory locations. Ex: age refers to a person's age, and birth refers to a person's date of birth.

A statement can set a variable to a value using the assignment operator (=). Note that this is different from the equal sign of mathematics. Ex: age = 6 or birth = "May 15". The left side of the assignment statement is a variable, and the right side is the value the variable is assigned.

Assigning and using variables Assigning and using variables; ch 1, video 5

Assigning and using variables

## Variable naming rules

A variable name can consist of letters, digits, and underscores and be of any length. The name cannot start with a digit. Ex: 101class is invalid. Also, letter case matters. Ex: Total is different from total. Python's style guide recommends writing variable names in snake case, which is all lowercase with underscores in between each word, such as first_name or total_price.

A name should be short and descriptive, so words are preferred over single characters in programs for readability. Ex: A variable named count indicates the variable's purpose better than a variable named c.

Python has reserved words, known as keywords, which have special functions and cannot be used as names for variables (or other objects).

Keywords False await else import pass None break except in raise True class finally is return and continue for lambda try as def from nonlocal while assert del global not with asynch elif if or yield

Valid variable names

Final score Write a Python computer program that: Creates a variable, team1, assigned with the value "Liverpool". Creates a variable, team2, assigned with the value "Chelsea". Creates a variable score1, assigned with the value 4. Creates a variable, score2, assigned with the value 3. Prints team1, "versus", and team2 as a single line of output. Prints "Final score: ", score1, "to", score2 as a single line of output.

# Page 4 | OpenStax Python Programming: Chapter Statements - String basics

*Module ID: m00005 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,227*

## Learning objectives

By the end of this section you should be able to

Use the built-in len() function to get a string's length. Concatenate string literals and variables using the + operator.

## Quote marks

A string is a sequence of characters enclosed by matching single (') or double (") quotes. Ex: "Happy birthday!" and '21' are both strings.

To include a single quote (') in a string, enclose the string with matching double quotes ("). Ex: "Won't this work?" To include a double quote ("), enclose the string with matching single quotes ('). Ex: 'They said "Try it!", so I did'.

Rules for strings. Valid string Invalid string "17" or '17' 17 "seventeen" or 'seventeen' seventeen "Where?" or 'Where?' "Where?' "I hope you aren't sad." 'I hope you aren't sad.' 'The teacher said "Correct!" ' "The teacher said "Correct!" "

Valid and invalid strings

## len() function

A common operation on a string object is to get the string length, or the number of characters in the string. The len() function, when called on a string value, returns the string length.

Using len() to get the length of a string Using len() to get the length of a string; ch 1, video 6

Applying len() function to string values

## Concatenation

Concatenation is an operation that combines two or more strings sequentially with the concatenation operator (+). Ex: "A" + "part" produces the string "Apart".

Concatenating multiple strings Concatenating multiple strings; ch 1, video 7

String concatenation

Name length Write a program that asks the user to input their first and last name separately. Use the following prompts (example input in bold): What is your first name? Alan What is your last name? Turing The program should then output the length of each name. Based on the example input above, the output would be: Your first name is 4 letters long Your last name is 6 letters long

Punctuation Write a Python computer program that: Assigns the string "Freda" to a variable, name. Assigns the string "happy" to a variable, feel. Prints the string "Hi Freda!" with a single print() function using the variable name. Prints the string "I'm glad you feel happy." with a single print() function using the variable feel.

# Page 5 | OpenStax Python Programming: Chapter Statements - Number basics

*Module ID: m00006 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,512*

## Learning objectives

By the end of this section you should be able to

Use arithmetic operators to perform calculations. Explain the precedence of arithmetic operators.

## Numeric data types

Python supports two basic number formats, integer and floating-point. An integer represents a whole number, and a floating-point format represents a decimal number. The format a language uses to represent data is called a data type. In addition to integer and floating-point types, programming languages typically have a string type for representing text.

Integer and floating-point Integer and floating point; ch 1, video 8

Integers, floats, and strings Assume that x = 1, y = 2.0, and s = "32".

## Basic arithmetic

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, and division.

Four basic arithmetic operators exist in Python: Addition (+) Subtraction (-) Multiplication (*) Division (/)

Examples of arithmetic operators Examples of arithmetic operators; ch 1, video 9

Applying arithmetic operators Assume that x = 7, y = 20, and z = 2.

## Operator precedence

When a calculation has multiple operators, each operator is evaluated in order of precedence. Ex: 1 + 2 * 3 is 7 because multiplication takes precedence over addition. However, (1 + 2) * 3 is 9 because parentheses take precedence over multiplication.

Operator precedence from highest to lowest. Operator Description Example Result () Parentheses (1 + 2) * 3 9 ** Exponentiation 2 ** 4 16 +, - Positive, negative -math.pi -3.14159 *, / Multiplication, division 2 * 3 6 +, - Addition, subtraction 1 + 2 3

Order of operations in an arithmetic expression Order of operations in an arithmetic expression; ch 1, video 10

Multiple arithmetic operators

Values and types Write a Python computer program that: Defines an integer variable named 'int_a' and assigns 'int_a' with the value 10. Defines a floating-point variable named 'float_a' and assigns 'float_a' with the value 10.0. Defines a string variable named 'string_a' and assigns 'string_a' with the string value "10". Prints the value of each of the three variables along with their type.

Meters to feet Write a Python computer program that: Assigns the integer value 10 to a variable, meters. Assigns the floating-point value 3.28 to a variable, meter2feet. Calculates 10 meters in feet by multiplying meter by meter2feet. Store the result in a variable, feet. Prints the content of variable feet in the output.

# Page 6 | OpenStax Python Programming: Chapter Statements - Error messages

*Module ID: m00007 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,935*

## Learning objectives

By the end of this section you should be able to

Identify the error type and line number in error messages. Correct syntax errors, name errors, and indentation errors.

## How to read errors

A natural part of programming is making mistakes. Even experienced programmers make mistakes when writing code. Errors may result when mistakes are made when writing code. The computer requires very specific instructions telling the computer what to do. If the instructions are not clear, then the computer does not know what to do and gives back an error.

When an error occurs, Python displays a message with the following information:

The line number of the error. The type of error (Ex: SyntaxError). Additional details about the error.

Ex: Typing print "Hello!" without parentheses is a syntax error. In Python, parentheses are required to use print. When attempting to run print "Hello!", Python displays the following error:

Traceback (most recent call last): File "/home/student/Desktop/example.py", line 1 print "Hello" ^ SyntaxError: Missing parentheses in call to 'print'. Did you mean print("Hello")?

The caret character (^) shows where Python found the error. Sometimes the error may be located one or two lines before where the caret symbol is shown because Python may not have discovered the error until then. Traceback is a Python report of the location and type of error. The word traceback suggests a programmer trace back in the code to find the error if the error is not seen right away.

Learning to read error messages carefully is an important skill. The amount of technical jargon can be overwhelming at first. But this information can be very helpful.

Incorrect variable name Incorrect variable name; ch 1, video 11

Parts of an error Given the following error message: Traceback (most recent call last): File "/home/student/Desktop/example.py", line 2 print "test" ^ SyntaxError: Missing parentheses in call to 'print'. Did you mean print("test")?

## Common types of errors

Different types of errors may occur when running Python programs. When an error occurs, knowing the type of error gives insight about how to correct the error. The following table shows examples of mistakes that anyone could make when programming.

Simple mistakes. Mistake Error message Explanation print("Have a nice day!" SyntaxError: unexpected EOF while parsing The closing parenthesis is missing. Python is surprised to reach the end of file (EOF) before this line is complete. word = input("Type a word: ) SyntaxError: EOL while scanning string literal The closing quote marks are missing. As a result, the string does not terminate before the end of line (EOL). print("You typed:", wird) NameError: name 'wird' is not defined The spelling of word is incorrect. The programmer accidentally typed the wrong key. prints("You typed:", word) NameError: name 'prints' is not defined The spelling of print is incorrect. The programmer accidentally typed an extra letter. print("Hello") IndentationError: unexpected indent The programmer accidentally typed a space at the start of the line. print("Goodbye") IndentationError: unexpected indent The programmer accidentally pressed the Tab key at the start of the line.

Types of errors For each program below, what type of error will occur?

Three errors The following program has three errors. Run the program to find the first error, and correct the corresponding line of code. Then run the program again to find and correct the second error. Keep running and correcting the program until no errors are found.

Wrong symbols This code is based on an earlier example, but the code contains several mistakes. One line is missing required punctuation, and another line uses incorrect symbols. Run the program to find the first error, and correct the corresponding line of code. Keep running and correcting the program until no errors are found.

# Page 7 | OpenStax Python Programming: Chapter Statements - Comments

*Module ID: m00008 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,137*

## Learning objectives

By the end of this section you should be able to

Write concise, meaningful comments that explain intended functionality of the code. Write a docstring (more verbose comment) that describes the program functionality.

## The hash character

Comments are short phrases that explain what the code is doing. Ex: Lines 1, 8, and 10 in the following program contain comments. Each comment begins with a hash character (#). All text from the hash character to the end of the line is ignored when running the program. In contrast, hash characters inside of strings are treated as regular text. Ex: The string "Item #1: " does not contain a comment.

When writing comments:

The # character should be followed by a single space. Ex: # End of menu is easier to read than #End of menu. Comments should explain the purpose of the code, not just repeat the code itself. Ex: # Get the user's preferences is more descriptive than # Input item1 and item2.

Program with three comments # Display the menu options print("Lunch Menu") print("----------") print("Burrito") print("Enchilada") print("Taco") print("Salad") print() # End of menu # Get the user's preferences item1 = input("Item #1: ") item2 = input("Item #2: ")

Simple comments

## Code quality

The example program above had two parts: (1) display the menu options, and (2) get the user's preferences. Together, the blank lines and comments show the overall structure of the program.

Programmers spend more time reading code than writing code. Therefore, making code easier for others to read and understand is important. Two ways to improve code quality include:

Separate each part (lines that have a similar purpose) with a blank line. Write a comment before each part. Not every line needs a comment.

Comments in a program Comments in a program; ch 1, video 12

Code quality

## Documentation

Python programs may optionally begin with a string known as a docstring. A docstring is documentation written for others who will use the program but not necessarily read the source code. Most of the official documentation at docs.python.org is generated from docstrings.

Documentation can be long, so docstrings are generally written as multi-line strings (""""). Common elements of a docstring include a one-line summary, a blank line, and a longer description.

Vacations docstring Vacations docstring; ch 1, video 13

Documentation

Whose birthday Add two comments to the following program: one for the input, and one for the output. Separate the input and output with a blank line. Then, compare your comments with the sample solution, and ask yourself the following questions: Are your comments longer or shorter? Why? Is the formatting of your comments correct?

Gravity calculation Write a docstring for the following program. The first line of the docstring should explain, in one short sentence, what the program is. The second line of the docstring should be blank. The third and subsequent lines should include a longer explanation of what the program does. At the end of the docstring, add a line that says "Author: " followed by your name.

# Page 8 | OpenStax Python Programming: Chapter Statements - Why Python?

*Module ID: m00009 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,568*

## Learning objectives

By the end of this section you should be able to

Name two historical facts about how Python was first created. Describe two ways Python is considered a popular language.

## Historical background

Python has an interesting history. In 1982, Guido van Rossum, the creator of Python, started working at CWI, a Dutch national research institute. He joined a team that was designing a new programming language, named ABC, for teaching and prototyping. ABC's simplicity was ideal for beginners, but the language lacked features required to write advanced programs.

Several years later, van Rossum joined a different team at CWI working on an operating system. The team needed an easier way to write programs for monitoring computers and analyzing data. Languages common in the 1980's were (and still are) difficult to use for these kinds of programs. van Rossum envisioned a new language that would have a simple syntax, like ABC, but also provide advanced features that professionals would need.

At first, van Rossum started working on this new language as a hobby during his free time. He named the language Python because he was a fan of the British comedy group Monty Python. Over the next year, he and his colleagues successfully used Python many times for real work. van Rossum eventually decided to share Python with the broader programming community online. He freely shared Python's entire source code so that anyone could write and run Python programs.

Python's first release, known as Version 0.9.0, appeared in 1991, about six years after C++ and four years before Java. van Rossum's decisions to make the language simple yet advanced, suitable for everyday tasks, and freely available online contributed to Python's long-term success.

Key decisions Key decisions; ch 1, video 14

Python history

Exploring further For more details about Python's history, see "A brief history of Python" by Vasilisa Sheromova, and "History and Philosophy of Python" by Bernd Klein.

## Popularity of Python

Over the years, Python has become a nonprofit organization with a thriving community. Millions of programmers around the world use Python for all kinds of interesting projects. Hundreds of thousands of Python libraries have been released as open source software. The Python community is very active and supportive online, answering questions and sharing code.

One way to see Python's popularity is the TIOBE index. TIOBE is a Dutch company that provides products and services for measuring software code quality. Since 2001, TIOBE has tracked the popularity of programming languages and posted the results online. shows the TIOBE index over time for five of the most popular languages.

The TIOBE index is based on the number of search engine results for each language. The percentage refers to how many results belong to that language. Python has been among the top 10 languages every year since 2004. In October 2021, Python became the #1 language on the TIOBE index. No other language but C and Java had been #1 for the previous 20 years.

Another way to see Python's popularity is to analyze how frequently Python is discussed online. Stack Overflow is a question-and-answer website for programmers. shows the number of questions asked each month that were tagged with Python, JavaScript, and so forth. In recent years, Python has become the most asked about language in programming forums.

TIOBE programming community index. Source: www.tiobe.com

Stack Overflow questions per month. Source: data.stackexchange.com

Python popularity

# Page 9 | OpenStax Python Programming: Chapter Statements - Chapter summary

*Module ID: m00114 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,586*

This chapter introduced the basics of programming in Python, including:

print() and input(). Variables and assignment. Strings, integers, and floats. Arithmetic, concatenation. Common error messages. Comments and docstrings.

At this point, you should be able to write programs that ask for input, perform simple calculations, and output the results. The programming practice below ties together most topics presented in the chapter.

Chapter 1 reference. Function Description print(values) Outputs one or more values, each separated by a space, to the user. input(prompt) If present, prompt is output to the user. The function then reads a line of input from the user. len(string) Returns the length (the number of characters) of a string. type(value) Returns the type (or class) of a value. Ex: type(123) is <class 'int'>. Operator Description = (Assignment) Assigns (or updates) the value of a variable. In Python, variables begin to exist when assigned for the first time. + (Concatenation) Appends the contents of two strings, resulting in a new string. + (Addition) Adds the values of two numbers. - (Subtraction) Subtracts the value of one number from another. * (Multiplication) Multiplies the values of two numbers. / (Division) Divides the value of one number by another. ** (Exponentiation) Raises a number to a power. Ex: 3**2 is three squared. Syntax Description # (Comment) All text is ignored from the # symbol to the end of the line. ' or " (String) Strings may be written using either kind of quote. Ex: 'A' and "A" represent the same string. By convention, this book uses double quotes (") for most strings. """ (Docstring) Used for documentation, often in multi-line strings, to summarize a program's purpose or usage.

Fun facts Write a program that assigns a variable named number to any integer of your choice. Ex: number = 74. Then, use this variable to calculate and output the following results: 74 squared is 5476 74 cubed is 405224 One tenth of 74 is 7.4 74 plus 123 is 197 74 minus 456 is -382 Run the program multiple times, using a different integer each time. Your output should be mathematically correct for any integer that you choose. The point of this exercise is to perform basic arithmetic within a print statement. Do not use any other variables besides number. Your program should have only one assignment statement (at the beginning).

Mad lib A mad lib is a word game in which one person asks others for words to substitute into a pre-written story. The story is then read aloud with the goal of making people laugh. This exercise is based the Vacations Mad Lib available on the Printables section of MadLibs.com. Write a program that asks the user to input two adjectives and two nouns (user input in bold): Adjective: tranquil Adjective: scandalous Noun: pancake Noun: field Use input() to display each prompt exactly as shown. The user's input should be on the same line as the prompt. Each colon must be followed by exactly one space. After reading the input, the program should output the following three lines: A vacation is when you take a trip to some tranquil place with your scandalous family. Usually you go to some place that is near a/an pancake or up on a/an field. Notice that the first line should be completely blank. Replace the bold words (from the above example) with the actual words input by the user. Your final program should have four input statements, three print statements, and at least two comments. For completeness, write an appropriate docstring at the top of the program.

# Page 10 | OpenStax Python Programming: Chapter Expressions - The Python shell

*Module ID: m00011 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,646*

## Learning objectives

By the end of this section you should be able to

Use a Python shell to run statements and expressions interactively. Explain the function of the up and down arrow keyboard shortcuts.

## The interpreter

Python is a high-level language, meaning that the source code is intended for humans to understand. Computers, on the other hand, understand only low-level machine code made up of 1's and 0's. Programs written in high-level languages must be translated into machine code to run. This translation process can happen all at once, or a little at a time, depending on the language.

Python is an interpreted language: the source code is translated one line at a time while the program is running. The Python interpreter translates source code into machine code and runs the resulting program. If and when an error occurs, the interpreter stops translating the source code and displays an error message.

Most development environments include a Python shell for experimenting with code interactively. A shell, also called a console or terminal, is a program that allows direct interaction with an interpreter. The interpreter usually runs an entire program all at once. But the interpreter can run one line of code at a time within a Python shell.

Running a Python shell Running a Python shell; ch 2, video 1

Using a Python shell

## The arrow keys

A Python shell is convenient for exploring and troubleshooting code. The user can try something, look at the results, and then try something else. When an error occurs, an error message is displayed, but the program keeps running. That way, the user can edit the previous line and correct the error interactively.

The acronym REPL (pronounced "rep ul") is often used when referring to a shell. REPL stands for "read-eval-print loop," which describes the repetitive nature of a shell:

Read/input some code Evaluate/run the code Print any results Loop back to step 1

Most shells maintain a history of every line of code the user types. Pressing the up or down arrow key on the keyboard displays the history. The up arrow displays the previous line; the down arrow displays the next line. That way, the user can repeat a line without having to type the line again.

Correcting a typo Correcting a typo; ch 2, video 2

Using the arrow keys

Exploring the shell Running code interactively is a great way to learn how Python works. Open a Python shell on your computer, or use the one at python.org/shell. Then enter any Python code, one line at a time, to see the result. Here are a few expressions to try: x = 5 3*x - 5 3 * (x-5) x type(1) type('1') str(1) int('1') abs(-5) abs(5) len("Yo") len("HoHo") round(9.49) round(9.50) Note: These functions (type, str, int, len, and round) will be explored in more detail later in the chapter. You can read more about the built-in functions in the Python documentation.

Correcting mistakes Open a Python shell on your computer, or use the one at python.org/shell. Run the following two statements in the shell: x = 123 y = 456 Making mistakes is common while typing in a shell. The following lines include typos and other errors. For each line: (1) run the line in a shell to see the result, (2) press the up arrow to repeat the line, and (3) edit the line to get the correct result. print("Easy as", X) print("y divided by 2 is", y / 0) name = intput("What is your name? ") print(name, "is", int(name), "letters long.") print("That's all folks!) The expected output, after correcting typos, should look like: Easy as 123 y divided by 2 is 228.0 (no error/output) Stacie is 6 letters long. That's all folks!

# Page 11 | OpenStax Python Programming: Chapter Expressions - Type conversion

*Module ID: m00012 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,636*

## Learning objectives

By the end of this section you should be able to

Explain how the interpreter uses implicit type conversion. Use explicit type conversion with int(), float(), and str().

## Implicit type conversion

Common operations update a variable such that the variable's data type needs to be changed. Ex: A GPS first assigns distance with 250, an integer. After a wrong turn, the GPS assigns distance with 252.5, a float. The Python interpreter uses implicit type conversion to automatically convert one data type to another. Once distance is assigned with 252.5, the interpreter will convert distance from an integer to a float without the programmer needing to specify the conversion.

Example: Book ratings Example: Book ratings; ch 2, video 3

Implicit type conversion in practice Consider the example above.

## Explicit type conversion

A programmer often needs to change data types to perform an operation. Ex: A program should read in two values using input() and sum the values. Remember input() reads in values as strings. A programmer can use explicit type conversion to convert one data type to another.

int() converts a data type to an integer. Any fractional part is removed. Ex: int(5.9) produces 5. float() converts a data type to a float. Ex: float(2) produces 2.0. str() converts a data type to a string. Ex: str(3.14) produces "3.14".

Example: Ordering pizza Example: Ordering pizza; ch 2, video 4

Example: Ordering pizza Consider the example above.

Using int(), float(), and str() Given x = 4.5 and y = int(x), what is the value of each expression?

Grade average The following program computes the average of three predefined exam grades and prints the average twice. Improve the program to read the three grades from input and print the average first as a float, and then as an integer, using explicit type conversion. Ignore any differences that occur due to rounding.

Cups of water The following program should read in the ounces of water the user drank today and compute the number of cups drank and the number of cups left to drink based on a daily goal. Assume a cup contains 8 ounces. Fix the code to calculate cups_drank and cups_left and match the following: ounces is an integer representing the ounces the user drank. cups_drank is a float representing the number of cups of water drank. cups_left is an integer representing the number of cups of water left to drink (rounded down) out of the daily goal of 8 cups.

Product as float The following program reads two integers in as strings. Calculate the product of the two integers, and print the result as a float.

# Page 12 | OpenStax Python Programming: Chapter Expressions - Mixed data types

*Module ID: m00013 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,387*

## Learning objectives

By the end of this section you should be able to

Identify the data types produced by operations with integers, floats, and strings. Use operators and type conversions to combine integers, floats, and strings.

## Combining integers and floats

Programmers often need to combine numbers of different data types. Ex: A program computes the total for an online shopping order:

quantity = int(input()) price = float(input()) total = quantity * price print(total)

quantity is an integer, and price is a float. So what is the data type of total? For input 3 and 5.0, total is a float, and the program prints 15.0.

Combining an integer and a float produces a float. A float is by default printed with at least one figure after the decimal point and has as many figures as needed to represent the value. Note: Division using the / operator always produces a float.

Operations combining integers and floats Operations combining integers and floats; ch 2, video 5

Operations combining integers and floats

## Combining numeric types and strings

Easy type conversion in Python can lead a programmer to assume that any data type can be combined with another. Ex: Noor's program reads in a number from input and uses the number in a calculation. This results in an error in the program because the input() function by default stores the number as a string. Strings and numeric data types are incompatible for addition, subtraction, and division. One of the operands needs to be explicitly converted depending on the goal of arithmetic or string concatenation.

The * operator also serves as the repetition operator, which accepts a string operand and an integer operand and repeats the string. Ex: "banjo" * 3 produces "banjobanjobanjo".

Adding a string and an integer Adding a string and an integer; ch 2, video 6

Operations combining numeric types and strings

After the point Write a program that reads in a string of digits that represents the digits after the decimal point of a number, num. Concatenate the input string together with '.' and num, and print the result. Ex: If input is 345, the program will print 2.345.

Print n times Write a program that reads in two strings, str1 and str2, and an integer, count. Concatenate the two strings with a space in between and a newline ("\n") at the end. Print the resulting string count times.

# Page 13 | OpenStax Python Programming: Chapter Expressions - Floating-point errors

*Module ID: m00014 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,623*

## Learning objectives

By the end of this section you should be able to

Explain numerical inaccuracies related to floating-point representation. Use the round() function to mitigate floating-point errors in output.

Computers store information using 0's and 1's. All information must be converted to a string of 0's and 1's. Ex: 5 is converted to 101. Since only two values, 0 or 1, are allowed the format is called binary.

Floating-point values are stored as binary by Python. The conversion of a floating point number to the underlying binary results in specific types of floating-point errors.

A round-off error occurs when floating-point values are stored erroneously as an approximation. The difference between an approximation of a value used in computation and the correct (true) value is called a round-off error.

Ex: Storing the float (0.1)10 results in binary values that actually produce (0.1000000000000000055511151231257827021181583404541015625)10 when converted back, which is not exactly equal to (0.1)10.

Round-off error. # Print floats with 30 decimal places print(f'{0.1:.30f}') # prints 0.1 print(f'{0.2:.30f}') # prints 0.2 print(f'{0.4:.30f}') # prints 0.4 0.100000000000000005551115123126 0.200000000000000011102230246252 0.400000000000000022204460492503 (The example above shows a formatted string or f-string, which are introduced in the Objects chapter.)

An overflow error occurs when a value is too large to be stored. The maximum and minimum floating-point values that can be represented are 1.8×10308 and –1.8×10308, respectively. Attempting to store a floating-point value outside the range (–1.8×10308,1.8×10308) leads to an overflow error.

Below, 3.0256 and 3.0512 can be represented, but 3.01024 is too large and causes an overflow error.

Overflow error. print('3.0 to the power of 256 =', 3.0**256) print('3.0 to the power of 512 = ', 3.0**512) print('3.0 to the power of 1024 = ', 3.0**1024) 3.0 to the power of 256 = 1.3900845237714473e+122 3.0 to the power of 512 = 1.9323349832288915e+244 3.0 to the power of 1024 = Traceback (most recent call last): File "<stdin>", line 3, in <module> print('3.0 to the power of 1024 = ', 3.0**1024) OverflowError: (34, 'Numerical result out of range')

Floating-point errors For each situation, which error occurs?

## Floating point round() function

Python's round() function is used to round a floating-point number to a given number of decimal places. The function requires two arguments. The first argument is the number to be rounded. The second argument decides the number of decimal places to which the number is rounded. If the second argument is not provided, the number will be rounded to the closest integer. The round() function can be used to mitigate floating-point errors.

Ex:

round(2.451, 2) = 2.45 round(2.451) = 2

Examples of round() function Examples of a round () function; ch 2, video 7

Examples of round() function

Inaccurate tips The following code calculates the tip amount, given a bill amount and the tip ratio. Experiment with the following bill amounts and tip ratios and see if any inaccuracies may result in calculating the tip amount. bill amount: 22.70 and 33.33 tip ratio: 0.15, 0.18, and 0.20

Area of a triangle Complete the following steps to calculate a triangle's area, and print the result of each step. The area of a triangle is bh2, where b is the base and h is the height. Calculate the area of a triangle with base = 7 and height = 3.5. Round the triangle's area to one decimal place. Round the triangle's area to the nearest integer value.

# Page 14 | OpenStax Python Programming: Chapter Expressions - Dividing integers

*Module ID: m00015 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 4,078*

## Learning objectives

By the end of this section you should be able to

Evaluate expressions that involve floor division and modulo. Use the modulo operator to convert between units of measure.

## Division and modulo

Python provides two ways to divide numbers:

True division (/) converts numbers to floats before dividing. Ex: 7 / 4 becomes 7.0 / 4.0, resulting in 1.75. Floor division (//) computes the quotient, or the number of times divided. Ex: 7 // 4 is 1 because 4 goes into 7 one time, remainder 3. The modulo operator (%) computes the remainder. Ex: 7 % 4 is 3.

Note: The % operator is traditionally pronounced "mod" (short for "modulo"). Ex: When reading 7 % 4 out loud, a programmer would say "seven mod four."

Quotient and remainder Quotient and remainder; ch 2, video 8

Division and modulo What is the value of each expression?

## Unit conversions

Division is useful for converting one unit of measure to another. To convert centimeters to meters, a variable is divided by 100. Ex: 300 centimeters divided by 100 is 3 meters.

Amounts often do not divide evenly as integers. 193 centimeters is 1.93 meters, or 1 meter and 93 centimeters. A program can use floor division and modulo to separate the units:

The quotient, 1 meter, is 193 // 100. The remainder, 93 centimeters, is 193 % 100.

Programs often use floor division and modulo together. If one line of code floor divides by m, the next line will likely modulo by m. The unit m by which an amount is divided is called the modulus. Ex: When converting centimeters to meters, the modulus is 100.

Money and time Money and time; ch 2, video 9

Unit conversions

Arrival time Having a mobile device can be a lifesaver on long road trips. Programs like Google Maps find the shortest route and estimate the time of arrival. The time of arrival is based on the current time plus how long the trip will take. Write a program that (1) inputs the current time and estimated length of a trip, (2) calculates the time of arrival, and (3) outputs the results in hours and minutes. Your program should use the following prompts (user input in bold): Current hour (0-23)? 13 Current minute (0-59)? 25 Trip time (in minutes)? 340 In this example, the current time is 13:25 (1:25pm). The trip time is 340 minutes (5 hours and 40 minutes). 340 minutes after 13:25 is 19:05 (7:05pm). Your program should output the result in this format: Arrival hour is 19 Arrival minute is 5 The arrival hour must be between 0 and 23. Ex: Adding 120 minutes to 23:00 should be 1:00, not 25:00. The arrival minute must be between 0 and 59. Ex: Adding 20 minutes to 8:55 should be 9:15, not 8:75. Hint: Multiply the current hour by 60 to convert hours to minutes. Then, calculate the arrival time, in total minutes, as an integer. Your code must not use Python keywords from later chapters, such as if or while. The solution requires only addition, multiplication, division, and modulo.

Change machine Self-checkout aisles are becoming increasingly popular at grocery stores. Customers scan their own items, and a computer determines the total purchase amount. Customers who pay in cash insert dollar bills, and a machine automatically dispenses change in coins. That's where this program comes into the story. Your task is to calculate how many of each coin to dispense. Your program should use the following prompts (user input in bold): Total amount? 18.76 Cash payment? 20 You may assume that the cash paid will always be a whole number (representing dollar bills) that is greater than or equal to the total amount. The program should calculate and output the amount of change due and how many dollars, quarters, dimes, nickels, and pennies should be dispensed: Change Due $1.24 Dollars: 1 Quarters: 0 Dimes: 2 Nickels: 0 Pennies: 4 Hint: Calculate the total change, in cents, as an integer. Use the round() function to avoid floating-point errors. Your code must not use Python keywords from later chapters, such as if or while. The solution requires only subtraction, multiplication, division, and modulo.

# Page 15 | OpenStax Python Programming: Chapter Expressions - The math module

*Module ID: m00016 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 4,476*

## Learning objectives

By the end of this section you should be able to

Distinguish between built-in functions and math functions. Use functions and constants defined in the math module.

## Importing modules

Python comes with an extensive standard library of modules. A module is previously written code that can be imported in a program. The import statement defines a variable for accessing code in a module. Import statements often appear at the beginning of a program.

The standard library also defines built-in functions such as print(), input(), and float(). A built-in function is always available and does not need to be imported. The complete list of built-in functions is available in Python's official documentation.

A commonly used module in the standard library is the math module. This module defines functions such as sqrt() (square root). To call sqrt(), a program must import math and use the resulting math variable followed by a dot. Ex: math.sqrt(25) evaluates to 5.0.

The following program imports and uses the math module, and uses built-in functions for input and output.

Calculating the distance between two points import math x1 = float(input("Enter x1: ")) y1 = float(input("Enter y1: ")) x2 = float(input("Enter x2: ")) y2 = float(input("Enter y2: ")) distance = math.sqrt((x2-x1)**2 + (y2-y1)**2) print("The distance is", distance)

Importing math in a Python shell Importing math in a Python shell; ch 2, video 10

Built-in functions and math module

## Mathematical functions

Commonly used math functions and constants are shown below. The complete math module listing is available in Python's official documentation.

Example constants in the math module. Constant Value Description math.e e=2.71828... Euler's number: the base of the natural logarithm. math.pi π=3.14159... The ratio of the circumference to the diameter of a circle. math.tau τ=6.28318... The ratio of the circumference to the radius of a circle. Tau is equal to 2π.

Example functions in the math module. Function Description Examples Number-theoretic math.ceil(x) The ceiling of x: the smallest integer greater than or equal to x. math.ceil(7.4) → 8 math.ceil(-7.4) → -7 math.floor(x) The floor of x: the largest integer less than or equal to x. math.floor(7.4) → 7 math.floor(-7.4) → -8 Power and logarithmic math.log(x) The natural logarithm of x (to base e). math.log(math.e) → 1.0 math.log(0) → ValueError: math domain error math.log(x, base) The logarithm of x to the given base. math.log(8, 2) → 3.0 math.log(10000, 10) → 4.0 math.pow(x, y) x raised to the power y. Unlike the ** operator, math.pow() converts x and y to type float. math.pow(3, 0) → 1.0 math.pow(3, 3) → 27.0 math.sqrt(x) The square root of x. math.sqrt(9) → 3.0 math.sqrt(-9) → ValueError: math domain error Trigonometric math.cos(x) The cosine of x radians. math.cos(0) → 1.0 math.cos(math.pi) → -1.0 math.sin(x) The sine of x radians. math.sin(0) → 0.0 math.sin(math.pi/2) → 1.0 math.tan(x) The tangent of x radians. math.tan(0) → 0.0 math.tan(math.pi/4) → 0.999 (Round-off error; the result should be 1.0.)

Using math functions and constants

Quadratic formula In algebra, a quadratic equation is written as ax2+bx+c=0. The coefficients a, b, and c are known values. The variable x represents an unknown value. Ex: 2x2+3x−5=0 has the coefficients a=2, b=3, and c=−5. The quadratic formula provides a quick and easy way to solve a quadratic equation for x: x= −b± b 2 −4ac 2a The plus-minus symbol indicates the equation has two solutions. However, Python does not have a plus-minus operator. To use this formula in Python, the formula must be separated: x 1 = −b+ b 2 −4ac 2a x 2 = −b− b 2 −4ac 2a Write the code for the quadratic formula in the program below. Test your program using the following values for a, b, and c: Provided input Expected output a b c x1 x2 1 0 -4 2.0 -2.0 1 2 -3 1.0 -3.0 2 1 -1 0.5 -1.0 0 1 1 division by zero 1 0 1 math domain error

Cylinder formulas In geometry, the surface area and volume of a right circular cylinder can be computed as follows: A=2πrh+2π r 2 V=π r 2 h Write the code for these two formulas in the program below. Hint: Your solution should use both math.pi and math.tau. Test your program using the following values for r and h: Provided input Expected output r h area volume 0 0 0.0 0.0 1 1 12.57 3.14 1 2 18.85 6.28 2.5 4.8 114.67 94.25 3.1 7.0 196.73 211.33 If you get an error, try to look up what that error means.

# Page 16 | OpenStax Python Programming: Chapter Expressions - Formatting code

*Module ID: m00017 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 6,585*

## Learning objectives

By the end of this section you should be able to

Identify good spacing for expressions and statements. Write multi-line statements using implicit line joining.

## Recommended spacing

Most spaces in Python code are ignored when running programs; however, spaces at the start of a line are very important. The following two programs are equivalent:

Good spacing: name = input("Enter someone's name: ") place = input("Enter a famous place: ") print(name, "should visit", place + "!") Poor spacing: name=input ("Enter someone's name: " ) place =input("Enter a famous place: ") print( name,"should visit" , place+ "!")

One might argue that missing or extra spaces do not matter. After all, the two programs above run exactly the same way. However, the "poor spacing" version is more difficult to read. Code like name=input and place+ might lead to confusion.

Good programmers write code that is as easy to read as possible. That way, other programmers are more likely to understand the code. To encourage consistency, the Python community has a set of guidelines about where to put spaces and blank lines, what to name variables, how to break up long lines, and other important topics.

Python style guide PEP 8 is the official style guide for Python. PEP stands for Python Enhancement Proposal. Members of the Python community write PEPs to document best practices and propose new features. The table below is based on guidelines from PEP 8 under the heading Whitespace in Expressions and Statements.

Guidelines for spaces. Guideline Example Common Mistakes Parentheses: no space before or after. print("Go team!") print ("Go team!") print( "Go team!" ) Commas: no space before, one space after. print("Hello", name) print("Hello" , name) print("Hello",name) Assignment: one space before and after the =. name = input("Your name? ") name=input("Your name? ") name= input("Your name? ") name =input("Your name? ") Concatenation: one space before and after the +. print("Hi", name + "!") print("Hi", name+"!") print("Hi", name+ "!") print("Hi", name +"!") Arithmetic: use space to show lower precedence. x**2 + 5*x - 8 x ** 2 + 5 * x - 8 x ** 2+5 * x-8 x**2+5*x-8

Recommended spacing

## Automatic concatenation

Long strings make Python programs difficult to read. Ex: This program prints the first sentence of the US Declaration of Independence:

print("The unanimous Declaration of the thirteen united States of America, When in the Course of human events, it becomes necessary for one people to dissolve the political bands which have connected them with another, and to assume among the powers of the earth, the separate and equal station to which the Laws of Nature and of Nature's God entitle them, a decent respect to the opinions of mankind requires that they should declare the causes which impel them to the separation.")

PEP 8 recommends that each line of code be less than 80 characters long. That way, programmers won't need to scroll horizontally to read the code. The above program can be rewritten by breaking up the original string:

print("The unanimous Declaration of the thirteen united States of " "America, When in the Course of human events, it becomes " "necessary for one people to dissolve the political bands " "which have connected them with another, and to assume among " "the powers of the earth, the separate and equal station to " "which the Laws of Nature and of Nature's God entitle them, a " "decent respect to the opinions of mankind requires that they " "should declare the causes which impel them to the separation.")

For convenience, Python automatically concatenates multiple strings. The + operator is not required in this situation.

String concatenation String concatenation; ch 2, video 11

String literal concatenation

## Multi-line statements

Most statements in a Python program need only one line of code. But occasionally longer statements need to span multiple lines. Python provides two ways to write multi-line statements:

Explicit line joining, using \ characters: decl = "The unanimous Declaration of the thirteen united States of " \ "America, When in the Course of human events, it becomes " \ "necessary for one people to dissolve the political bands..." Implicit line joining, using parentheses: decl = ("The unanimous Declaration of the thirteen united States of " "America, When in the Course of human events, it becomes " "necessary for one people to dissolve the political bands...")

Implicit line joining is more common, since many statements and expressions use parentheses anyway. PEP 8 recommends avoiding the use of explicit line joining whenever possible.

Multi-line statements

Spaced out The following code works correctly but is formatted poorly. In particular, the code does not include spaces recommended by PEP 8. Furthermore, two of the lines are about 90 characters long. Reformat the code to follow the guidelines in this section. Be careful not to change the behavior of the code itself.

Five quotes Write a program that prints the following five quotes (source: BrainyQuote) from Guido van Rossum, the creator of Python. Your program should have exactly five print statements, one for each quote: 1. "If you're talking about Java in particular, Python is about the best fit you can get amongst all the other languages. Yet the funny thing is, from a language point of view, JavaScript has a lot in common with Python, but it is sort of a restricted subset." 2. "The second stream of material that is going to come out of this project is a programming environment and a set of programming tools where we really want to focus again on the needs of the newbie. This environment is going to have to be extremely user-friendly." 3. "I have this hope that there is a better way. Higher-level tools that actually let you see the structure of the software more clearly will be of tremendous value." 4. "Now, it's my belief that Python is a lot easier than to teach to students programming and teach them C or C++ or Java at the same time because all the details of the languages are so much harder. Other scripting languages really don't work very well there either." 5. "I would guess that the decision to create a small special purpose language or use an existing general purpose language is one of the toughest decisions that anyone facing the need for a new language must make." Notice that all of these lines are longer than 80 characters, and some contain single quote marks. Format the code using multi-line statements and escape sequences as necessary.

# Page 17 | OpenStax Python Programming: Chapter Expressions - Python careers

*Module ID: m00018 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,512*

## Learning objectives

By the end of this section you should be able to

Summarize how Python is used in fields other than CS and IT. Describe two different kinds of applications made with Python.

## Fields and applications

Learning Python opens the door to many programming-related careers. Example job titles include software engineer, data scientist, web developer, and systems analyst. These jobs often require a degree in computer science (CS), information technology (IT), or a related field. However, programming is not limited to these fields and careers.

Many professionals use Python to support the work they do. Python programs can automate tasks and solve problems quickly. shows a few examples of Python outside of computing fields. Knowing how to program is a useful skill that can enhance any career.

Python is a versatile language that supports many kinds of applications. shows a few examples of programs that can be written in Python. Given Python's usefulness and popularity, Python is a great language to learn. A supportive community of professionals and enthusiasts is ready to help.

Python outside of CS and IT. Field Example use of Python Business An accountant writes a Python program to generate custom sales reports. Education A teacher writes a Python program to organize homework submissions. Fine arts An artist writes a Python program to operate an interactive art display. Humanities A linguist writes a Python program to analyze changes in slang word usage. Science A biologist writes a Python program to analyze DNA sequences for cancer.

Applications built with Python. Application Example use of Python Artificial intelligence An engineer develops models to support image and voice recognition. Ex: TensorFlow library. Data visualization A statistician creates charts to make sense of large amounts of data. Ex: Matplotlib library. General purpose Most programs in this book are general. Ex: Read inputs, perform calculations, print results. Scientific computing Python is very useful for conducting experiments and analyzing data. Ex: The SciPy project. Web development Python can run interactive websites. Ex: Instagram is built with Django, a Python framework.

Fields and applications

Exploring further For more examples of applications that can be built with Python, see "Top 12 Fascinating Python Applications in Real-World" by Rohit Sharma. For more information about Python related careers, see "What Does a Python Developer Do?" in BrainStation's career guide.

# Page 18 | OpenStax Python Programming: Chapter Expressions - Chapter summary

*Module ID: m00019 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,555*

Highlights from this chapter include:

Expressions and statements can be run interactively using a shell. Input strings can be converted to other types. Ex: int(input()). Strings can be concatenated with other types. Ex: "$" + str(cost). Floats are subject to round-off and overflow errors. Integers can be divided exactly using // and %. Modules like math provide many useful functions. Formatting long lines helps improve readability.

At this point, you should be able to write programs that ask for input of mixed types, perform mathematical calculations, and output results with better formatting. The programming practice below ties together most topics presented in the chapter.

Chapter 2 reference. Function Description abs(x) Returns the absolute value of x. int(x) Converts x (a string or float) to an integer. float(x) Converts x (a string or integer) to a float. str(x) Converts x (a float or integer) to a string. round(x, ndigits) Rounds x to ndigits places after the decimal point. If ndigits is omitted, returns the nearest integer to x. Operator Description s * n (Repetition) Creates a string with n copies of s. Ex: "Ha" * 3 is "HaHaHa". x / y (Real division) Divides x by y and returns the entire result as a float. Ex: 7 / 4 is 1.75. x // y (Floor division) Divides x by y and returns the quotient as an integer. Ex: 7 // 4 is 1. x % y (Modulo) Divides x by y and returns the remainder as an integer. Ex: 7 % 4 is 3.

Baking bread The holidays are approaching, and you need to buy ingredients for baking many loaves of bread. According to a recipe by King Arthur Flour, you will need the following ingredients for each loaf: 1 1/2 teaspoons instant yeast 1 1/2 teaspoons salt 1 1/2 teaspoons sugar 2 1/2 cups all-purpose flour 2 cups sourdough starter 1/2 cup lukewarm water Write a program that inputs the following variables: bread_weight (float), serving_size (float), and num_guests (int). The output will look like the following: Note: The measures the program comes up with are exact, but to bake, the baker would have to use some approximation. Ex: 9.765625 cups all-purpose flour really means 9 and 3/4 cups. For 25 people, you will need 3.90625 loaves of bread: 5.859375 teaspoons instant yeast 5.859375 teaspoons salt 5.859375 teaspoons sugar 9.765625 cups all-purpose flour 7.8125 cups sourdough starter 1.953125 cups lukewarm water In the above output, bread_weight is 16.0 ounces, serving_size is 2.5 ounces, and num_guests is 25 people. Use these three variables to calculate the number of loaves needed. Make sure your output matches the above example exactly. Notice that each line of the ingredients begins with two spaces.

Tip calculator Google has a variety of search tricks that present users with instant results. If you search on Google for tip calculator, an interactive tool is included at the top of the results. The goal of this exercise is to implement a similar tip calculator. Begin by prompting the user to input the following values (user input in bold): Enter bill amount: 43.21 Percentage to tip: 18 Number of people: 2 Then calculate the tip amount and total amount for the bill, based on the user input. Output the results using this format: Tip amount: $7.78 Total amount: $50.99 Tip per person: $3.89 Total per person: $25.49 Your program should output all dollar amounts rounded to two decimal places. The output should be exactly six lines, as shown above. Notice the blank line before each section of the output. Notice also the space before but not after the dollar sign.

# Page 19 | OpenStax Python Programming: Chapter Decisions - Boolean values

*Module ID: m00028 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,273*

## Learning objectives

By the end of this section you should be able to

Explain a Boolean value. Use bool variables to store Boolean values. Demonstrate converting integers, floats, and strings to Booleans. Demonstrate converting Booleans to integers, floats, and strings. Use comparison operators to compare integers, floats, and strings.

## bool data type

People often ask binary questions such as yes/no or true/false questions. Ex: Do you like pineapple on pizza? Ex: True or false: I like pineapple on pizza. The response is a Boolean value, meaning the value is either true or false. The bool data type, standing for Boolean, represents a binary value of either true or false. true and false are keywords, and capitalization is required.

Example: Crosswalk sign Example: Crosswalk sign; ch 4, video 1

Using Boolean variables Consider the following code: is_fruit = "True" is_vegetable = 0 is_dessert = False

## Type conversion with bool()

Deciding whether a value is true or false is helpful when writing programs/statements based on decisions. Converting data types to Booleans can seem unintuitive at first. Ex: Is "ice cream" True? But the conversion is actually simple.

bool() converts a value to a Boolean value, True or False. True: any non-zero number, any non-empty string False: 0, empty string

Converting integers, floats, and strings using bool() Converting integers, floats, and strings using bool(); ch 4, video 2

Converting numeric types and strings to Booleans

Converting Booleans to numeric types and strings Given is_on = True, what is the value of each expression?

## Comparison operators

Programmers often have to answer questions like "Is the current user the admin?" A programmer may want to compare a string variable, user, to the string, "admin". Comparison operators are used to compare values, and the result is either true or false. Ex: is_admin = (user == "admin"). user is compared with "admin" using the == operator, which tests for equality. The Boolean variable, is_admin, is assigned with the Boolean result.

The 6 comparison operators: equal to: == not equal to: != greater than: > less than: < greater than or equal to: >= less than or equal to: <=

Example: Rolling a d20 in a tabletop game Example: Rolling a d20 in a tabletop game; ch 4, video 3

Comparing values For each new variable, what is the value of compare_result?

= vs == A common mistake is using = for comparison instead of ==. Ex: is_zero = num=0 will always assign is_zero and num with 0, regardless of num's original value. The = operator performs assignment and will modify the variable. The == operator performs comparison, does not modify the variable, and produces True or False.

Exploring further Unicode Basic Latin Chart

Friday Boolean "It's Friday, I'm in love" —from "Friday I'm in Love," a song released by the Cure in 1992. Write a program that reads in the day of the week. Assign the Boolean variable, in_love, with the result of whether the day is Friday or not.

Even numbers Write a program that reads in an integer and prints whether the integer is even or not. Remember, a number is even if the number is divisible by 2. To test this use number % 2 == 0. Ex: If the input is 6, the output is "6 is even: True".

# Page 20 | OpenStax Python Programming: Chapter Decisions - If-else statements

*Module ID: m00029 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,961*

## Learning objectives

By the end of this section you should be able to

Identify which operations are performed when a program with if and if-else statements is run. Identify the components of an if and if-else statement and the necessary formatting. Create an if-else statement to perform an operation when a condition is true and another operation otherwise.

## if statement

If the weather is rainy, grab an umbrella! People make decisions based on conditions like if the weather is rainy, and programs perform operations based on conditions like a variable's value. Ex: A program adds two numbers. If the result is negative, the program prints an error.

A condition is an expression that evaluates to true or false. An if statement is a decision-making structure that contains a condition and a body of statements. If the condition is true, the body is executed. If the condition is false, the body is not executed.

The if statement's body must be grouped together and have one level of indentation. The PEP 8 style guide recommends four spaces per indentation level. The Python interpreter will produce an error if the body is empty.

Example: Quantity check Example: Quantity check; ch 4, video 4

Using boolean variables A Boolean variable already has a value of True or False and can be used directly in a condition rather than using the equality operator. Ex: if is_raining == True: can be simplified to if is_raining:.

Using if statements

## if-else statement

An if statement defines actions to be performed when a condition is true. What if an action needs to be performed only when the condition is false? Ex: If the restaurant is less than a mile away, we'll walk. Else, we'll drive.

An else statement is used with an if statement and contains a body of statements that is executed when the if statement's condition is false. When an if-else statement is executed, one and only one of the branches is taken. That is, the body of the if or the body of the else is executed. Note: The else statement is at the same level of indentation as the if statement, and the body is indented.

if-else statement template:

# Statements before if condition: # Body else: # Body # Statements after

Example: Trivia question Example: Trivia question; ch 4, video 5

Exploring if-else statements

Improved division The following program divides two integers. Division by 0 produces an error. Modify the program to read in a new denominator (with no prompt) if the denominator is 0.

Converting temperature units The following program reads in a temperature as a float and the unit as a string: "f" for Fahrenheit or "c" for Celsius.Calculate new_temp, the result of converting temp from Fahrenheit to Celsius or Celsius to Fahrenheit based on unit. Calculate new_unit: "c" if unit is "f" and "f" if unit is "c". Conversion formulas: Degrees Celsius = (degrees Fahrenheit - 32) * 5/9 Degrees Fahrenheit = (degrees Celsius * 5/9) + 32

# Page 21 | OpenStax Python Programming: Chapter Decisions - Boolean operations

*Module ID: m00030 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,110*

## Learning objectives

By the end of this section you should be able to

Explain the purpose of logical operators. Describe the truth tables for and, or, and not. Create expressions with logical operators. Interpret if-else statements with conditions using logical operators.

## Logical operator: and

Decisions are often based on multiple conditions. Ex: A program printing if a business is open may check that hour >= 9 and hour < 17. A logical operator takes condition operand(s) and produces True or False.

Python has three logical operators: and, or, and not. The and operator takes two condition operands and returns True if both conditions are true.

Truth table: p and q. p q p and q True True True True False False False True False False False False

Example: Museum entry Example: Museum entry; ch 4, video 6

Using the and operator

## Logical operator: or

Sometimes a decision only requires one condition to be true. Ex: If a student is in the band or choir, they will perform in the spring concert. The or operator takes two condition operands and returns True if either condition is true.

Truth table: p or q. p q p or q True True True True False True False True True False False False

Example: Streaming prompt Example: Streaming prompt; ch 4, video 7

Using the or operator

## Logical operator: not

If the computer is not on, press the power button. The not operator takes one condition operand and returns True when the operand is false and returns False when the operand is true.

not is a useful operator that can make a condition more readable and can be used to toggle a Boolean's value. Ex: is_on = not is_on.

Truth table: not p. p not p True False False True

Example: Diving warning Example: Diving warning; ch 4, video 8

Using the not operator

Speed limits Write a program that reads in a car's speed as an integer and checks if the car's speed is within the freeway limits. A car's speed must be at least 45 mph but no greater than 70 mph on the freeway. If the speed is within the limits, print "Good driving". Else, print "Follow the speed limits".

# Page 22 | OpenStax Python Programming: Chapter Decisions - Operator precedence

*Module ID: m00031 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,782*

## Learning objectives

By the end of this section you should be able to

Describe how precedence impacts order of operations. Describe how associativity impacts order of operations. Explain the purpose of using parentheses in expressions with multiple operators.

## Precedence

When an expression has multiple operators, which operator is evaluated first? Precedence rules provide the priority level of operators. Operators with the highest precedence execute first. Ex: 1 + 2 * 3 is 7 because multiplication takes precedence over addition. However, (1 + 2) * 3 is 9 because parentheses take precedence over multiplication.

Operator precedence from highest to lowest. Operator Meaning () Parentheses ** Exponentiation (right associative) *, /, //, % Multiplication, division, floor division, modulo +, - Addition, subtraction <, <=, >, >=, ==, != Comparison operators not Logical not operator and Logical and operator or Logical or operator

Operator precedence Operator precedence; ch 4, video 9

Precedence rules Which part of each expression is evaluated first?

## Associativity

What if operators beside each other have the same level of precedence? Associativity determines the order of operations when precedence is the same. Ex: 8 / 4 * 3 is evaluated as (8/4) * 3 rather than 8 / (4*3) because multiplication and division are left associative. Most operators are left associative and are evaluated from left to right. Exponentiation is the main exception (noted above) and is right associative: that is, evaluated from right to left. Ex: 2 ** 3 ** 4 is evaluated as 2 ** (3**4).

When comparison operators are chained, the expression is converted into the equivalent combination of comparisons and evaluated from left to right. Ex. 10 < x <= 20 is evaluated as 10 < x and x <= 20.

Operation precedence Operation precedence; ch 4, video 10

Associativity How is each expression evaluated?

## Enforcing order and clarity with parentheses

Operator precedence rules can be hard to remember. Parentheses not only assert a different order of operations but also reduce confusion.

Using parentheses Using parenthesis; ch 4, video 11

Using parentheses

PEP 8 recommendations: spacing around operators The PEP 8 style guide recommends consistent spacing around operators to avoid extraneous and confusing whitespace. Avoid multiple spaces and an unequal amount of whitespace around operators with two operands. Avoid: x= y * 44 Better: x = y * 44 Avoid spaces immediately inside parentheses. Avoid: x = ( 4 * y ) Better: x = (4 * y) Surround the following operators with one space: assignment, augment assignment, comparison, Boolean. Avoid: x= y<44 Better: x = y < 44 Consider adding whitespace around operators with lower priority. Avoid: x = 5 * z+20 Better: x = 5*z + 20

# Page 23 | OpenStax Python Programming: Chapter Decisions - Chained decisions

*Module ID: m00032 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,687*

## Learning objectives

By the end of this section you should be able to

Identify the branches taken in an if-elif and if-elif-else statement. Create a chained decision statement to evaluate multiple conditions.

## elif

Sometimes, a complicated decision is based on more than a single condition. Ex: A travel planning site reviews the layovers on an itinerary. If a layover is greater than 24 hours, the site should suggest accommodations. Else if the layover is less than one hour, the site should alert for a possible missed connection.

Two separate if statements do not guarantee that only one branch is taken and might result in both branches being taken. Ex: The program below attempts to add a curve based on the input test score. If the input is 60, both if statements are incorrectly executed, and the resulting score is 75.

score = int(input()) if score < 70: score += 10 # Wrong: if 70 <= score < 85: score += 5

Chaining decision statements with elif allows the programmer to check for multiple conditions. An elif (short for else if) statement checks a condition when the prior decision statement's condition is false. An elif statement is part of a chain and must follow an if (or elif) statement.

if-elif statement template:

# Statements before if condition: # Body elif condition: # Body # Statements after

Example: Livestream features Example: Livestream features; ch 4, video 12

Using elif

## if-elif-else statements

Elifs can be chained with an if-else statement to create a more complex decision statement. Ex: A program shows possible chess moves depending on the piece type. If the piece is a pawn, show moving forward one (or two) places. Else if the piece is a bishop, show diagonal moves. Else if . . . (finish for the rest of the pieces).

Example: Possible chess moves Example: Possible chess moves; ch 4, video 13

Using elif within if-elif-else statements

Crochet hook size conversion Write a program that reads in a crochet hook's US size and computes the metric diameter in millimeters. (A subset of sizes is used.) If the input does not match B-G, the diameter should be assigned with -1.0. Ex: If the input is D, the output is "3.25 mm". Size conversions for US size: mm B : 2.25 C : 2.75 D : 3.25 E : 3.5 F : 3.75 G : 4.0

Color wavelengths Write a program that reads in an integer representing a visible light wavelength in nanometers. Print the corresponding color using the following inclusive ranges: Violet: 380–449 Blue: 450–484 Cyan: 485–499 Green: 500–564 Yellow: 565–589 Orange: 590–624 Red: 625–750 Assume the input is within the visible light spectrum, 380-750 inclusive. Given input: 550 The output is: Green

# Page 24 | OpenStax Python Programming: Chapter Decisions - Nested decisions

*Module ID: m00033 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 1,943*

## Learning objectives

By the end of this section you should be able to

Describe the execution paths of programs with nested if-else statements. Implement a program with nested if-else statements.

## Nested decision statements

Suppose a programmer is writing a program that reads in a game ID and player count and prints whether the user has the right number of players for the game.

The programmer may start with:

if game == 1 and players < 2: print("Not enough players") if game == 1 and players > 4: print("Too many players") if game == 1 and (2 <= players <= 4): print("Ready to start") if game == 2 and players < 3: print("Not enough players") if game == 2 and players > 6: print("Too many players") if game == 2 and (3 <= players <= 6): print("Ready to start")

The programmer realizes the code is redundant. What if the programmer could decide the game ID first and then make a decision about players? Nesting allows a decision statement to be inside another decision statement, and is indicated by an indentation level.

An improved program:

if game == 1: if players < 2: print("Not enough players") elif players > 4: print("Too many players") else: print("Ready to start") if game == 2: if players < 3: print("Not enough players") elif players > 6: print("Too many players") else: print("Ready to start") # Test game IDs 3-end

Example: Poisonous plant identification Example: Poisonous plant identification; ch 4, video 14

Using nested if-else statements

Meal orders Write a program that reads in a string, "lunch" or "dinner", representing the menu choice, and an integer, 1, 2, or 3, representing the user's meal choice. The program then prints the user's meal choice. Lunch Meal Options 1: Caesar salad 2: Spicy chicken wrap 3: Butternut squash soup Dinner Meal Options 1: Baked salmon 2: Turkey burger 3: Mushroom risotto Ex: If the input is: lunch 3 The output is: Your order: Butternut squash soup

# Page 25 | OpenStax Python Programming: Chapter Decisions - Chapter summary

*Module ID: m00035 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,739*

Highlights from this chapter include:

Booleans represent a value of True or False. Comparison operators compare values and produce True or False. Logical operators take condition operand(s) and produce True or False. Operators are evaluated in order according to precedence and associativity. Conditions are expressions that evaluate to True or False. Decision statements allow different paths of execution (branches) through code based on conditions. Decision statements can be nested inside other decision statements. Conditional expressions are single-line versions of if-else statements.

At this point, you should be able to write programs that evaluate conditions and execute code statements accordingly with the correct order of operations. The programming practice below ties together most topics presented in the chapter.

Chapter 4 reference. Function Description bool(x) Converts x to a Boolean value, either True or False. Operator Description x == y (Equality) Compares the values of x and y and returns True if the values are equal and False otherwise. Ex: 10 == 10 is True. x != y (Inequality) Compares the values of x and y and returns True if the values are inequal and False otherwise. Ex: 7 != 4 is True. x > y (Greater than) Compares the values of x and y and returns True if the x is greater than y and False otherwise. Ex: 9 > 3 is True. x < y (Less than) Compares the values of x and y and returns True if the x is less than y and False otherwise. Ex: 9 < 8 is False. x >= y (Greater than or equal) Compares the values of x and y and returns True if the x is greater than or equal to y and False otherwise. Ex: 2 >= 2 is True. x <= y (Less than or equal) Compares the values of x and y and returns True if the x is less than or equal to y and False otherwise. Ex: 8 <= 7 is False. x and y (Logical) Evaluates the Boolean values of x and y and returns True if both are true. Ex: True and False is False. x or y (Logical) Evaluates the Boolean values of x and y and returns True if either is true. Ex: True or False is True. not x (Logical) Evaluates the Boolean value of x and returns True if the value is false and False if the value is true. Ex: not True is False. Decision statement Description if statement # Statements before if condition: # Body # Statements after else statement # Statements before if condition: # Body else: # Body # Statements after elif statement # Statements before if condition: # Body elif condition: # Body else: # Body # Statements after Nested if statement # Statements before if condition: if condition: # Body else: # Body else: if condition: # Body else: # Body # Statements after Conditional expression expression_if_true if condition else expression_if_false

# Page 26 | OpenStax Python Programming: Chapter Loops - While loop

*Module ID: m00037 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 3,087*

## Learning objectives

By the end of this section you should be able to

Explain the loop construct in Python. Use a while loop to implement repeating tasks.

A while loop is a code construct that runs a set of statements, known as the loop body, while a given condition, known as the loop expression, is true. At each iteration, once the loop statement is executed, the loop expression is evaluated again.

If true, the loop body will execute at least one more time (also called looping or iterating one more time). If false, the loop's execution will terminate and the next statement after the loop body will execute.

While loop While loop; ch 5, video 1

While loop example Fibonacci is a series of numbers in which each number is the sum of the two preceding numbers. The Fibonacci sequence starts with two ones: 1, 1, 2, 3, . . . . Consider the following code that prints all Fibonacci numbers less than 20, and answer the following questions. # Initializing the first two Fibonacci numbers f = 1 g = 1 print (f, end = ' ') # Running the loop while the last Fibonacci number is less than 20 while g < 20: print(g, end = ' ') # Calculating the next Fibonacci number and updating the last two sequence numbers temp = f f = g g = temp + g

## Counting with a while loop

A while loop can be used to count up or down. A counter variable can be used in the loop expression to determine the number of iterations executed. Ex: A programmer may want to print all even numbers between 1 and 20. The task can be done by using a counter initialized with 1. In each iteration, the counter's value is increased by one, and a condition can check whether the counter's value is an even number or not. The change in the counter's value in each iteration is called the step size. The step size can be any positive or negative value. If the step size is a positive number, the counter counts in ascending order, and if the step size is a negative number, the counter counts in descending order.

A program printing all odd numbers between 1 and 10 # Initialization counter = 1 # While loop condition while counter <= 10: if counter % 2 == 1: print(counter) # Counting up and increasing counter's value by 1 in each iteration counter += 1

Counting with while loop Counting with while loop; ch 5, video 2

while loop counting examples Given the code, answer the following questions. n = 4 while n > 0: print(n) n = n - 1 print("value of n after the loop is", n)

Reading inputs in a while loop Write a program that takes user inputs in a while loop until the user enters "begin". Test the code with the given input values to check that the loop does not terminate until the input is "begin". Once the input "begin" is received, print "The while loop condition has been met.". Enter different input words to see when the while loop condition is met.

Sum of odd numbers Write a program that reads two integer values, n1 and n2. Use a while loop to calculate the sum of odd numbers between n1 and n2 (inclusive of n1 and n2). Remember, a number is odd if number % 2 != 0.

# Page 27 | OpenStax Python Programming: Chapter Loops - For loop

*Module ID: m00038 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,976*

## Learning objectives

By the end of this section you should be able to

Explain the for loop construct. Use a for loop to implement repeating tasks.

In Python, a container can be a range of numbers, a string of characters, or a list of values. To access objects within a container, an iterative loop can be designed to retrieve objects one at a time. A for loop iterates over all elements in a container. Ex: Iterating over a class roster and printing students' names.

For loop example for iterating over a container object For loop example for iterating over a container object; ch 5, video 3

For loop over a string container A string variable can be considered a container of multiple characters, and hence can be iterated on. Given the following code, answer the questions. str_var = "A string" count = 0 for c in str_var: count += 1 print(count)

A for loop can be used for iteration and counting. The range() function is a common approach for implementing counting in a for loop. A range() function generates a sequence of integers between the two numbers given a step size. This integer sequence is inclusive of the start and exclusive of the end of the sequence. The range() function can take up to three input values. Examples are provided in the table below.

Using the range() function. Range function Description Example Output range(end) Generates a sequence beginning at 0 until end. Step size: 1 range(4) 0, 1, 2, 3 range(start, end) Generates a sequence beginning at start until end. Step size: 1 range(0, 3) 0, 1, 2 range(2, 6) 2, 3, 4, 5 range(-13, -9) -13, -12, -11, -10 range(start, end, step) Generates a sequence beginning at start until end. Step size: step range(0, 4, 1) 0, 1, 2, 3 range(1, 7, 2) 1, 3, 5 range(3, -2, -1) 3, 2, 1, 0, -1 range(10, 0, -4) 10, 6, 2

Two programs printing all integer multiples of 5 less than 50 (Notice the compactness of the for construction compared to the while) # For loop condition using # range() function to print # all multiples of 5 less than 50 for i in range(0, 50, 5): print(i) # While loop implementation of printing # multiples of 5 less than 50 # Initialization i = 0 # Limiting the range to be less than 50 while i < 50: print(i) i+=5

For loop using a range() function

Counting spaces Write a program using a for loop that takes in a string as input and counts the number of spaces in the provided string. The program must print the number of spaces counted. Ex: If the input is "Hi everyone", the program outputs 1.

Sequences Write a program that reads two integer values, n1 and n2, with n1 < n2, and performs the following tasks: Prints all even numbers between the two provided numbers (inclusive of both), in ascending order. Prints all odd numbers between the two provided numbers (exclusive of both), in descending order. Input: 2 8 Prints: 2 4 6 8 7 5 3 Note: the program should return an error message if the second number is smaller than the first.

# Page 28 | OpenStax Python Programming: Chapter Loops - Nested loops

*Module ID: m00039 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,477*

## Learning objectives

By the end of this section you should be able to

Implement nested while loops. Implement nested for loops.

A nested loop has one or more loops within the body of another loop. The two loops are referred to as outer loop and inner loop. The outer loop controls the number of the inner loop's full execution. More than one inner loop can exist in a nested loop.

Nested while loops Nested while loops; ch 5, video 5

Available appointments Consider a doctor's office schedule. Each appointment is 30 minutes long. A program to print available appointments can use a nested for loop where the outer loop iterates over the hours, and the inner loop iterates over the minutes. This example prints time in hours and minutes in the range between 8:00am and 10:00am. In this example, the outer loop iterates over the time's hour portion between 8 and 9, and the inner loop iterates over the time's minute portion between 0 and 59. hour = 8 minute = 0 while hour <= 9: while minute <= 59: print(hour, ":", minute) minute += 30 hour += 1 minute = 0 The above code's output is: 8 : 0 8 : 30 9 : 0 9 : 30

Nested while loop question set

## Nested for loops

A nested for loop can be implemented and used in the same way as a nested while loop. A for loop is a preferable option in cases where a loop is used for counting purposes using a range() function, or when iterating over a container object, including nested situations. Ex: ■■Iterating over multiple course rosters. The outer loop iterates over different courses, and the inner loop iterates over the names in each course roster.

Nested for loops Nested for loops; ch 5, video 4

Nested loop practices

Mixed loops The two for and while loop constructs can also be mixed in a nested loop construct. Ex: Printing even numbers less than a given number in a list. The outer loop can be implemented using a for loop iterating over the provided list, and the inner loop iterates over all even numbers less than a given number from the list using a while loop. numbers = [12, 5, 3] i = 0 for n in numbers: while i < n: print (i, end = " ") i += 2 i = 0 print()

Printing a triangle of numbers Write a program that prints the following output: 1 2 3 4 5 6 7 8 9 1 2 3 4 5 6 7 8 1 2 3 4 5 6 7 1 2 3 4 5 6 1 2 3 4 5 1 2 3 4 1 2 3 1 2 1 Finish!

Printing two triangles Write a program that prints the following output using nested while and for loops: **** *** ** * ++++ +++ ++ +

# Page 29 | OpenStax Python Programming: Chapter Loops - Break and continue

*Module ID: m00040 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,410*

## Learning objectives

By the end of this section you should be able to

Analyze a loop's execution with break and continue statements. Use break and continue control statements in while and for loops.

## Break

A break statement is used within a for or a while loop to allow the program execution to exit the loop once a given condition is triggered. A break statement can be used to improve runtime efficiency when further loop execution is not required.

Ex: A loop that looks for the character "a" in a given string called user_string. The loop below is a regular for loop for going through all the characters of user_string. If the character is found, the break statement takes execution out of the for loop. Since the task has been accomplished, the rest of the for loop execution is bypassed.

user_string = "This is a string." for i in range(len(user_string)): if user_string[i] == 'a': print("Found at index:", i) break

Break statement in a while loop Break statement in a while loop; ch 5, video 6

Infinite loop A break statement is an essential part of a loop that does not have a termination condition. A loop without a termination condition is known as an infinite loop. Ex: An infinite loop that counts up starting from 1 and prints the counter's value while the counter's value is less than 10. A break condition is triggered when the counter's value is equal to 10, and hence the program execution exits. counter = 1 while True: if counter >= 10: break print(counter) counter += 1

Using a break statement

## Continue

A continue statement allows for skipping the execution of the remainder of the loop without exiting the loop entirely. A continue statement can be used in a for or a while loop. After the continue statement's execution, the loop expression will be evaluated again and the loop will continue from the loop's expression. A continue statement facilitates the loop's control and readability.

Continue statement in a while loop Control statement in a while loop; ch 5, video 7

Using a continue statement

Using break control statement in a loop Write a program that reads a string value and prints "Found" if the string contains a space character. Else, prints "Not found".

Using a continue statement in a loop Complete the following code so that the program calculates the sum of all the numbers in list my_list that are greater than or equal to 5.

# Page 30 | OpenStax Python Programming: Chapter Loops - Loop else

*Module ID: m00041 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,007*

## Learning objectives

By the end of this section you should be able to

Use loop else statement to identify when the loop execution is interrupted using a break statement. Implement a loop else statement with a for or a while loop.

A loop else statement runs after the loop's execution is completed without being interrupted by a break statement. A loop else is used to identify if the loop is terminated normally or the execution is interrupted by a break statement.

Ex: A for loop that iterates over a list of numbers to find if the value 10 is in the list. In each iteration, if 10 is observed, the statement "Found 10!" is printed and the execution can terminate using a break statement. If 10 is not in the list, the loop terminates when all integers in the list are evaluated, and hence the else statement will run and print "10 is not in the list." Alternatively, a Boolean variable can be used to track whether number 10 is found after loop's execution terminates.

Finding the number 10 in a list In the code examples below, the code on the left prints "Found 10!" if the variable i's value is 10. If the value 10 is not in the list, the code prints "10 is not in the list.". The code on the right uses the seen Boolean variable to track if the value 10 is in the list. After loop's execution, if seen's value is still false, the code prints "10 is not in the list.". numbers = [2, 5, 7, 11, 12] for i in numbers: if i == 10: print("Found 10!") break else: print("10 is not in the list.") numbers = [2, 5, 7, 11, 12] seen = False for i in numbers: if i == 10: print("Found 10!") seen = True if seen == False: print("10 is not in the list.")

Loop else template Loop else template; ch 5, video 8

Loop else practices

Sum of values less than 10 Write a program that, given a list, calculates the sum of all integer values less than 10. If a value greater than or equal to 10 is in the list, no output should be printed. Test the code for different values in the list.

# Page 31 | OpenStax Python Programming: Chapter Loops - Chapter summary

*Module ID: m00042 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,514*

Highlights from this chapter include:

A while loop runs a set of statements, known as the loop body, while a given condition, known as the loop expression, is true. A for loop can be used to iterate over elements of a container object. A range() function generates a sequence of integers between the two numbers given a step size. A nested loop has one or more loops within the body of another loop. A break statement is used within a for or a while loop to allow the program execution to exit the loop once a given condition is triggered. A continue statement allows for skipping the execution of the remainder of the loop without exiting the loop entirely. A loop else statement runs after the loop's execution is completed without being interrupted by a break statement.

At this point, you should be able to write programs with loop constructs. The programming practice below ties together most topics presented in the chapter.

Chapter 5 reference. Function Description range(end) Generates a sequence beginning at 0 until end with step size of 1. range(start, end) Generates a sequence beginning at start until end with step size of 1. range(start, end, s) Generates a sequence beginning at start until end with the step size of s. Loop constructs Description while loop # initialization while expression: # loop body # statements after the loop for loop # initialization for loop_variable in container: # loop body # statements after the loop Nested while loop while outer_loop_expression: # outer loop body (1) while inner_loop_expression: # inner loop body # outer loop body (2) # statements after the loop break statement # initialization while loop_expression: # loop body if break_condition: break # remaining body of loop # statements after the loop continue statement # initialization while loop_expression: # loop body if continue_condition: continue # remaining body of loop # statements after the loop Loop else statement # initialization for loop_expression: # loop body if break_condition: break # remaining body of loop else: # loop else statement # statements after the loop

Prime numbers Write a program that takes in a positive integer number (N) and prints out the first N prime numbers on separate lines. Note: A prime number is a number that is not divisible by any positive number larger than 1. To check whether a number is prime, the condition of number % i != 0 can be checked for i greater than 1 and less than number. Ex: if N = 6, the output is: 2 3 5 7 11 13

# Page 32 | OpenStax Python Programming: Chapter Functions - Defining functions

*Module ID: m00044 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,279*

## Learning objectives

By the end of this section you should be able to

Identify function calls in a program. Define a parameterless function that outputs strings. Describe benefits of using functions.

## Calling a function

Throughout the book, functions have been called to perform tasks. Ex: print() prints values, and sqrt() calculates the square root. A function is a named, reusable block of code that performs a task when called.

Example: Simple math program Example: Simple math program; ch 6, video 1

Identifying function calls

## Defining a function

A function is defined using the def keyword. The first line contains def followed by the function name (in snake case), parentheses (with any parameters—discussed later), and a colon. The indented body begins with a documentation string describing the function's task and contains the function statements. A function must be defined before the function is called.

Example: Welcome message function Example: Welcome message function; ch 6, video 2

Defining functions

## Benefits of functions

A function promotes modularity by putting code statements related to a single task in a separate group. The body of a function can be executed repeatedly with multiple function calls, so a function promotes reusability. Modular, reusable code is easier to modify and is shareable among programmers to avoid reinventing the wheel.

Improving a program with a function Improving a program with a function; ch 6, video 3

Improving programs with functions Consider the code above.

Cinema concession stand Write a function, concessions(), that prints the food and drink options at a cinema. Given: concessions() The output is: Food/Drink Options: Popcorn: $8-10 Candy: $3-5 Soft drink: $5-7

Terms and conditions prompt Write a function, terms(), that asks the user to accept the terms and conditions, reads in Y/N, and outputs a response. In the main program, read in the number of users and call terms() for each user. Given inputs 1 and "Y", the output is: Do you accept the terms and conditions? Y Thank you for accepting. Given inputs 2, "N", and "Y", the output is: Do you accept the terms and conditions? N Have a good day. Do you accept the terms and conditions? Y Thank you for accepting.

# Page 33 | OpenStax Python Programming: Chapter Functions - Control flow

*Module ID: m00045 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,448*

## Learning objectives

By the end of this section you should be able to

Identify the control flow of a program. Describe how control flow moves between statements and function calls.

## Control flow and functions

Control flow is the sequence of program execution. A program's control flow begins at the main program but rarely follows a strict sequence. Ex: Control flow skips over lines when a conditional statement isn't executed.

When execution reaches a function call, control flow moves to where the function is defined and executes the function statements. Then, control flow moves back to where the function was called and continues the sequence.

Calling a brunch menu function Calling a brunch menu function; ch 6, video 4

Following the control flow

## Functions calling functions

Functions frequently call other functions to keep the modularity of each function performing one task. Ex: A function that calculates an order total may call a function that calculates sales tax. When a function called from another function finishes execution, control flow returns to the calling function.

Example: Book club email messages Example: Book club email messages; ch 6, video 5

Functions calling functions Consider the book club example above.

Updated terms and conditions prompt Write an updated function, terms(), that asks the user to accept the terms and conditions, reads in Y/N, and outputs a response by calling accepted() or rejected(). accepted() prints "Thank you for accepting the terms." and rejected() prints "You have rejected the terms. Thank you.". Given inputs 2, "Y" and "N", the output is: Do you accept the terms and conditions? Y Thank you for accepting the terms. Given a function call to terms() and input "N", the output is: Do you accept the terms and conditions? N You have rejected the terms. Thank you.

Laundromat information Write a program that uses three functions to print information about a laundromat, Liam's Laundry: laundromat_info(): Prints the name, Liam's Laundry, and hours of operation, 7a - 11p, and calls washers_open() and dryers_open() washers_open(): Reads an integer, assigns washer_count with the value, and prints washer_count dryers_open(): Reads an integer, assigns dryer_count with the value, and prints dryer_count The main program should just call laundromat_info(). Given inputs 50 and 40, the output is: Liam's Laundry 7a - 11p Open washers: 50 Open dryers: 40

# Page 34 | OpenStax Python Programming: Chapter Functions - Variable scope

*Module ID: m00046 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 2,889*

## Learning objectives

By the end of this section you should be able to

Identify the scope of a program's variables. Discuss the impact of a variable's scope.

## Global scope

A variable's scope is the part of a program where the variable can be accessed. A variable created outside of a function has global scope and can be accessed anywhere in the program. A Python program begins in global scope, and the global scope lasts for the entire program execution.

Global variables in a program with a function Global variables in a program with a function; ch 6, video 6

Global variables

## Local scope

A variable created within a function has local scope and only exists within the function. A local variable cannot be accessed outside of the function in which the variable was created. After a function finishes executing, the function's local variables no longer exist.

Global and local variables in a program with a function Global and local variables in a program with a function; ch 6, video 7

Local variables

## Using local and global variables together

Python allows global and local variables to have the same name, which can lead to unexpected program behavior. A function treats a variable edited within the function as a local variable unless told otherwise. To edit a global variable inside a function, the variable must be declared with the global keyword.

Editing global variables in a program with a function Editing global variables in a program with a function; ch 6, video 8

Using both local and global variables Consider the following variations on the example program with the input 9.

Benefits of limiting scope A programmer might ask, "Why not just make all variables global variables to avoid access errors?" Making every variable global can make a program messy. Ex: A programmer debugging a large program discovers a variable has the wrong value. If the whole program can modify the variable, then the bug could be anywhere in the large program. Limiting a variable's scope to only what's necessary and restricting global variable use make a program easier to debug, maintain, and update.

Battle royale game launch Write a program that reads in a selected game mode and calls one of two functions to launch the game. If the input is "br", call battle_royale(). Otherwise, call practice(). battle_royale(): Reads in the number of players. Computes the number of teammates still needed. A full team is 3 players. Calls the function find_teammates() with the calculated number. Prints "Match starting . . .". practice(): Reads in a string representing the desired map. Prints "Launching practice on [desired map]". Note: find_teammates() is provided and does not need to be edited. Given input: br 1 The output is: Finding 2 players... Match starting... Given input: p Queen's Canyon The output is: Launching practice on Queen's Canyon

# Page 35 | OpenStax Python Programming: Chapter Functions - Parameters

*Module ID: m00047 | Source: OpenStax (CC BY-NC-SA 4.0) | Characters: 4,354*

## Learning objectives

By the end of this section you should be able to

Identify a function's arguments and parameters. Describe how mutability affects how a function can modify arguments.

## Arguments and parameters

What if a programmer wants to write a function that prints the contents of a list? Good practice is to pass values directly to a function rather than relying on global variables. A function argument is a value passed as input during a function call. A function parameter is a variable representing the input in the function definition. Note: The terms "argument" and "parameter" are sometimes used interchangeably in conversation and documentation.

Global and local variables in a program with a function More global and local variables in a program with a function; ch 6, video 9

Arguments and parameters Consider the following: def print_welcome(name): print(f"Welcome {name}!") username = int(input("Enter new username: ")) print_welcome(username)

## Multiple arguments and parameters

Functions can have multiple parameters. Ex: A function uses two parameters, length and width, to compute the square footage of a room. Function calls must use the correct order and number of arguments to avoid undesired behavior and errors (unless using optional or keyword arguments as discussed later).

Using multiple arguments in a function call def print_div(op_1, op_2): """ Prints division operation """ print(f"{op_1}/{op_2} = {op_1/op_2}") num_1 = 6 num_2 = 3 print_div(num_1, num_2) # Prints "6/3 = 2.0" print_div(num_2, num_1) # Prints "3/6 = 0.5" print_div(num_1) # Error: Missing argument: op_2

Multiple arguments and parameters Consider the following: def calc_distance(x1, y1, x2, y2): dist = math.sqrt((x2-x1)**2 + (y2-y1)**2) print(dist) p1_x =int(input("Enter point 1's x: ")) p1_y =int(input("Enter point 1's y: ")) p2_x =int(input("Enter point 2's x: ")) p2_y =int(input("Enter point 2's y: ")) calc_distance(p1_x, p1_y, p2_x, p2_y)

## Modifying arguments and mutability

In Python, a variable is a name that refers to an object stored in memory, aka an object reference, so Python uses a pass-by-object-reference system. If an argument is changed in a function, the changes are kept or lost depending on the object's mutability. A mutable object can be modified after creation. A function's changes to the object then appear outside the function. An immutable object cannot be modified after creation. So a function must make a local copy to modify, and the local copy's changes don't appear outside the function.

Programmers should be cautious of modifying function arguments as these side effects can make programs difficult to debug and maintain.

Converting temperatures What are the values of weekend_temps and type after convert_temps() finishes? def convert_temps(temps, unit): if unit == "F": for i in range(len(temps)): temps[i] = (temps[i]-32) * 5/9 unit = "C" else: for i in range(len(temps)): temps[i] = (temps[i]*9/5) + 32 unit = "F" # Weekend temperatures in Fahrenheit. wknd_temps = [49.0, 51.0, 44.0] deg_sign = u"\N{DEGREE SIGN}" # Unicode metric = "F" # Convert from Fahrenheit to Celsius. convert_temps(wknd_temps, metric) for temp in wknd_temps: print(f"{temp:.2f}{deg_sign}{metric}", end=" ") The output is 9.44°F 10.56°F 6.67°F. type was changed to "C" in the function but didn't keep the change outside of the function. Why is the list argument change kept and not the string argument change? (Hint: A list is mutable. A string is immutable.)

Exploring a faulty function Exploring a faulty function; ch 6, video 10

Mutability and function arguments

Printing right triangle area Write a function, print_area(), that takes in the base and height of a right triangle and prints the triangle's area. The area of a right triangle is bh2, where b is the base and h is the height. Given input: 3 4 The output is: Triangle area: 6.0

Curving scores Write a function, print_scores(), that takes in a list of test scores and a number representing how many points to add. For each score, print the original score and the sum of the score and bonus. Make sure not to change the list. Given function call: print_scores([67, 68, 72, 71, 69], 10) The output is: 67 would be updated to 77 68 would be updated to 78 72 would be updated to 82 71 would be updated to 81 69 would be updated to 79